

핸즈온 랩: Microsoft Fabric에서 데이터 과학 시작하기

이 랩에서는 데이터를 수집하고, Notebook에서 데이터를 탐색하고, Data Wrangler로 데이터를 처리하고, 두 가지 유형의 모델을 학습시킵니다. 이 모든 단계를 수행함으로써 Microsoft Fabric의 데이터 과학 기능을 탐색할 수 있습니다.

이 랩을 완료하면 기계 학습 및 모델 추적에 대한 실무 경험을 얻고, Microsoft Fabric에서 *Notebooks*, *Data Wrangler*, *Experiments*, *Models*로 작업하는 방법을 배우게 됩니다.

이 실습을 완료하는 데 약 **20분**이 소요됩니다.

참고: 이 실습을 완료하려면 [Microsoft Fabric 평가판](#)이 필요합니다.

Workspace 만들기

Fabric에서 데이터 작업을 시작하기 전에 Fabric 평가판이 활성화된 Workspace를 만들어야 합니다.

- 웹 브라우저에서 [Microsoft Fabric 홈페이지](https://app.fabric.microsoft.com/home?experience=fabric) `https://app.fabric.microsoft.com/home?experience=fabric` 로 이동하여 Fabric 자격 증명으로 로그인합니다.
- 왼쪽 메뉴 모음에서 **Workspaces** 아이콘()을 선택합니다.
- 원하는 이름으로 새 Workspace를 만듭니다. **Advanced** 섹션에서 Fabric 용량(*Trial*, *Premium*, 또는 *Fabric*)을 포함하는 라이선스 모드를 선택해야 합니다.
- 새 Workspace가 열리면 처음에는 비어 있어야 합니다.

Notebook 만들기

코드를 실행하기 위해 **Notebook**을 만들 수 있습니다. Notebook은 여러 언어로 코드를 작성하고 실행할 수 있는 대화형 환경을 제공합니다.

- 왼쪽 메뉴 모음에서 **Create**를 선택합니다. *New* 페이지의 *Data Science* 섹션에서 **Notebook**을 선택하고, 원하는 고유한 이름을 지정합니다.
- 첫 번째 셀(현재 *code* 셀)을 선택한 다음, 오른쪽 상단의 동적 도구 모음에서 **M↓** 버튼을 사용하여 셀을 *markdown* 셀로 변환합니다.
-  (Edit) 버튼을 사용하여 셀을 편집 모드로 전환한 다음, 내용을 삭제하고 다음 텍스트를 입력합니다.

```
# Data science in Microsoft Fabric
```

데이터 가져오기

이제 코드를 실행하여 데이터를 가져오고 모델을 학습시킬 준비가 되었습니다. Azure Open Datasets의 [당뇨병 데이터 세트](#)를 사용합니다. 데이터를 로드한 후에는 데이터를 행과 열로 작업하기 위한 일반적인 구조인 Pandas 데이터프레임으로 변환합니다.

- Notebook에서 최신 셀 출력 아래의 **+ Code** 아이콘을 사용하여 새 코드 셀을 추가합니다.

2. 새 코드 셀에 다음 코드를 입력합니다.

```
# Azure storage access info for open dataset diabetes
blob_account_name = "azureopendatastorage"
blob_container_name = "mlsamples"
blob_relative_path = "diabetes"
blob_sas_token = r"" # Blank since container is Anonymous access

# Set Spark config to access blob storage
wasbs_path = f"wasbs://{s}%s.blob.core.windows.net/%s" % (blob_container_name,
blob_account_name, blob_relative_path)
spark.conf.set("fs.azure.sas.%s.%s.blob.core.windows.net" % (blob_container_name,
blob_account_name), blob_sas_token)
print("Remote blob path: " + wasbs_path)

# Spark read parquet, note that it won't load any data yet by now
df = spark.read.parquet(wasbs_path)
```

코드 설명: 이 코드는 공개적으로 액세스 가능한 Azure Blob Storage에 저장된 당뇨병 데이터 세트에 연결하기 위한 설정을 구성합니다. `spark.read.parquet(wasbs_path)` 는 지정된 경로의 Parquet 파일을 읽어 Spark DataFrame으로 로드합니다.

3. 셀 왼쪽의 ▶ **Run cell** 버튼을 사용하여 코드를 실행합니다.

4. 새 코드 셀을 추가하고 다음 코드를 입력하여 DataFrame의 내용을 확인합니다.

```
display(df)
```

5. 결과로 렌더링된 테이블 상단의 **Table**과 **+ New chart** 두 탭 중에서 **+ New chart**를 선택합니다.

6. 차트 오른쪽의 **Build my own** 옵션을 선택하여 새 시각화를 만듭니다.

7. 다음 차트 설정을 선택합니다.

- **Chart Type:** `Box plot`
- **Y-axis:** `y`

8. 레이블 옆 `y`의 분포를 보여주는 출력을 검토합니다.

그래프 구조 설명

- 상자(box): Q1(87)부터 Q3(212)까지 범위를 나타내며, 데이터의 중간 50%를 포함합니다.
- 세로선(중앙선): Median (140.5)를 나타냅니다.
- 수염(whisker): 최소값(25)과 최대값(346)을 연결합니다.
- 데이터가 왼쪽으로 조금 더 치우쳐진 모습입니다 (왼쪽 수염이 더 짧음), 즉 우측으로 길게 퍼진 분포를 암시합니다 (right-skewed distribution).

사용 목적

Box Plot은 다음을 빠르게 파악할 수 있어요:

- 데이터의 중앙 경향
- 퍼짐 정도

- 비대칭성 (skewness)
- 이상치 존재 여부

데이터 준비하기

개념 설명: Data Wrangler

Data Wrangler는 Microsoft Fabric에 내장된 데이터 준비 도구입니다. 코드를 작성하지 않고도 시각적인 인터페이스를 통해 데이터를 정리하고 변환하는 작업을 수행할 수 있습니다. 사용자가 UI에서 수행한 각 변환 단계는 자동으로 재사용 가능한 Python 코드로 생성되어 Notebook에 추가됩니다. 이는 데이터 전처리 작업을 가속화하고, 반복적인 작업을 자동화하는 데 매우 유용합니다.

1. 데이터는 Spark 데이터프레임으로 로드되었습니다. Data Wrangler는 Spark 또는 Pandas 데이터프레임을 모두 수용하지만, 현재는 Pandas에 최적화되어 있습니다. 따라서 데이터를 Pandas 데이터프레임으로 변환합니다. Notebook에서 다음 코드를 실행합니다.

```
df = df.toPandas()
df.head()
```

2. Notebook 리본에서 **Data Wrangler**를 선택한 다음, **df** 데이터 세트를 선택합니다. Data Wrangler가 실행되면 **Summary** 패널에 데이터프레임의 기술적 개요가 생성됩니다.
3. 현재 레이블 열은 연속형 변수인 **y**입니다. **y**를 예측하는 회귀 모델을 학습시킬 수 있지만, 그 결과 값은 해석하기 어려울 수 있습니다. 대신, 당뇨병 발병 위험이 낮은지 높은지를 예측하는 분류 모델을 학습시키는 것을 탐색해 보겠습니다. 분류 모델을 학습시키려면 **y** 값에 기반한 이진(binary) 레이블 열을 만들어야 합니다.
4. Data Wrangler에서 **y** 열을 선택합니다. 히스토그램에서 **220-240** 구간에서 빈도가 감소하는 것을 확인하세요. 75번째 백분위수인 **211.5**가 대략 두 영역의 전환 지점과 일치합니다. 이 값을 저위험과 고위험의 임계값으로 사용합니다.
5. **Operations** 패널로 이동하여 **Formulas**를 확장하고, **Create column from formula**를 선택합니다.
6. 다음 설정으로 새 열을 만듭니다.
 - **Column name:** **Risk**
 - **Column formula:** `(df['Y'] > 211.5).astype(int)`
7. **Apply**를 선택합니다.
8. 미리보기에 추가된 새 열 **Risk**를 검토합니다. 값이 **1**인 행의 수가 전체 행의 약 25%인지 확인합니다 (**y**의 75번째 백분위수이므로).
9. **Add code to notebook**을 선택합니다.
10. Data Wrangler에 의해 생성된 코드가 있는 셀을 실행합니다.
11. 새 셀에서 다음 코드를 실행하여 **Risk** 열이 예상대로 구성되었는지 확인합니다.

```
df_clean.describe()
```

머신러닝 모델 학습하기

개념 설명: MLflow

MLflow는 기계 학습 수명 주기를 관리하기 위한 오픈 소스 플랫폼입니다. Microsoft Fabric은 MLflow와 완벽하게 통합되어 있어, 모델 학습 과정을 자동으로 추적하고 관리할 수 있습니다.

- **Experiment:** 특정 목표(예: 당뇨병 회귀 예측)를 위한 모든 모델 학습 실행(run)을 그룹화하는 단위입니다.
- **Run:** 모델을 한 번 학습시키는 과정입니다. 각 실행마다 사용된 파라미터, 성능 메트릭, 생성된 모델 파일(아티팩트) 등이 자동으로 기록됩니다.
- **autolog():** MLflow의 자동 로깅 기능입니다. 이 함수를 호출하면 scikit-learn과 같은 일반적인 라이브러리로 모델을 학습시킬 때, 사용자가 수동으로 코드를 작성하지 않아도 파라미터, 메트릭, 모델 등을 자동으로 기록해 줍니다.

핸즈온의 의미: 이 섹션에서는 동일한 데이터셋으로 두 가지 다른 유형의 문제(회귀, 분류)를 해결하는 모델을 학습시킵니다.

MLflow를 사용하여 각 모델의 학습 과정을 별도의 실험(Experiment)으로 추적하고, 나중에 이 기록들을 비교하여 최적의 모델을 선택하는 전체 데이터 과학 워크플로우를 경험합니다.

회귀 모델 학습하기

1. 데이터를 훈련 및 테스트 데이터 세트로 분할하고, 예측하려는 레이블 `y`와 특징(feature)을 분리하기 위해 다음 코드를 실행합니다.

```
from sklearn.model_selection import train_test_split

X, y = df_clean[['AGE', 'SEX', 'BMI', 'BP', 'S1', 'S2', 'S3', 'S4', 'S5', 'S6']].values,
df_clean['Y'].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,
random_state=0)
```

2. 새 코드 셀을 추가하고 다음 코드를 실행하여 `diabetes-regression`이라는 MLflow 실험을 만듭니다.

```
import mlflow
experiment_name = "diabetes-regression"
mlflow.set_experiment(experiment_name)
```

3. 새 코드 셀을 추가하고 다음 코드를 실행하여 선형 회귀(Linear Regression) 모델을 학습시킵니다.

```
from sklearn.linear_model import LinearRegression

with mlflow.start_run():
    mlflow.autolog()

    model = LinearRegression()
    model.fit(X_train, y_train)
```

분류 모델 학습하기

1. 데이터를 훈련 및 테스트 데이터 세트로 분할하고, 예측하려는 레이블 `Risk`와 특징을 분리하기 위해 다음 코드를 실행합니다.

```
from sklearn.model_selection import train_test_split

X, y = df_clean[['AGE', 'SEX', 'BMI', 'BP', 'S1', 'S2', 'S3', 'S4', 'S5', 'S6']].values,
df_clean['Risk'].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,
random_state=0)
```

2. 새 코드 셀을 추가하고 다음 코드를 실행하여 `diabetes-classification` 이라는 MLflow 실험을 만듭니다.

```
import mlflow
experiment_name = "diabetes-classification"
mlflow.set_experiment(experiment_name)
```

3. 새 코드 셀을 추가하고 다음 코드를 실행하여 로지스틱 회귀(Logistic Regression) 모델을 학습시킵니다.

```
from sklearn.linear_model import LogisticRegression

with mlflow.start_run():
    mlflow.sklearn.autolog()

    model = LogisticRegression(C=1/0.1, solver="liblinear").fit(X_train, y_train)
```

실험(Experiments) 탐색하기

Microsoft Fabric은 모든 실험을 추적하고 시각적으로 탐색할 수 있게 해줍니다.

1. 왼쪽 허브 메뉴 바에서 Workspace로 이동합니다.
2. `diabetes-regression` 실험을 선택하여 엽니다.
3. **Run metrics**를 검토하여 회귀 모델의 정확도를 탐색합니다.
4. Workspace로 다시 이동하여 `diabetes-classification` 실험을 선택하여 엽니다.
5. **Run metrics**를 검토하여 분류 모델의 정확도를 탐색합니다. 다른 유형의 모델을 학습시켰기 때문에 메트릭 유형이 다른 점에 유의하세요.

모델 저장하기

실험 전반에 걸쳐 학습한 머신러닝 모델을 비교한 후, 가장 성능이 좋은 모델을 선택할 수 있습니다. 가장 성능이 좋은 모델을 사용하려면 모델을 저장하고 예측을 생성하는 데 사용합니다.

1. 실험 리본에서 **Save as ML model**을 선택합니다.
2. 새로 열린 팝업 창에서 **Create a new ML model**을 선택합니다.
3. `model` 폴더를 선택합니다.
4. 모델 이름을 `model-diabetes`로 지정하고 **Save**를 선택합니다.
5. 모델이 생성되면 화면 오른쪽 상단에 나타나는 알림에서 **View ML model**을 선택합니다. 저장된 모델은 **ML model versions** 아래에 연결됩니다.

모델, 실험, 그리고 실험 실행이 연결되어 있어 모델이 어떻게 학습되었는지 검토할 수 있습니다.

Notebook 저장 및 Spark 세션 종료하기

이제 모델 학습 및 평가를 마쳤으므로, 의미 있는 이름으로 Notebook을 저장하고 Spark 세션을 종료할 수 있습니다.

1. Notebook 메뉴 바에서  **Settings** 아이콘을 사용하여 Notebook 설정을 봅니다.
2. Notebook의 **Name**을 **Train and compare models**로 설정한 다음 설정 창을 닫습니다.
3. Notebook 메뉴에서 **Stop session**을 선택하여 Spark 세션을 종료합니다.

리소스 정리

이 실습에서는 Notebook을 생성하고 머신러닝 모델을 학습시켰습니다. scikit-learn을 사용하여 모델을 학습시키고 MLflow를 사용하여 성능을 추적했습니다.

모델 및 실험 탐색을 마쳤다면 이 실습을 위해 만든 Workspace를 삭제할 수 있습니다.

1. 왼쪽 막대에서 Workspace 아이콘을 선택하여 포함된 모든 항목을 봅니다.
2. 툴바의 ... 메뉴에서 **Workspace settings**를 선택합니다.
3. **General** 섹션에서 **Remove this workspace**를 선택합니다.